

컴퓨터 비전

Computer Vision

- Feature Extraction and Matching -

동아대학교 소프트웨어대학 AI학과
2026년 1학기

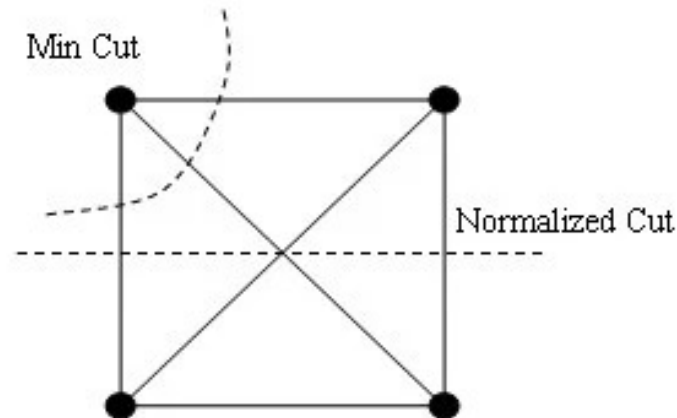
임 한 신

최적화 분할

- 영상의 그래프 표현
 - Normalized cut(N-cut, 정규화 절단)알고리즘
 - cut은 영상을 두 영역으로 분할했을 때 분할의 좋은 정도를 측정해 주는 목적 함수
 - C_1 과 C_2 가 클수록 둘 사이에 간선이 많아 cut은 덩달아 커지므로 cut을 사용한 분할 알고리즘은 영역을 자잘하게 분할하는 경향이 있음

$$cut(C_1, C_2) = \sum_{v_p \in C_1, v_q \in C_2} s_{pq}$$

- N-cut은 cut을 정규화하여 영역의 크기에 중립이 되게 해줌
- Spectral clustering 기법을 영상에 적용



대응점 문제(correspondence problem)

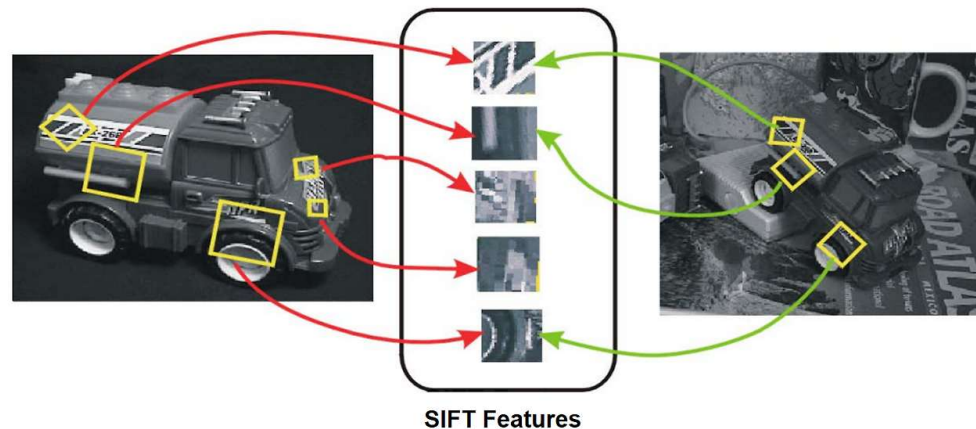
- 이웃한 영상에 나타난 같은 물체의 같은 곳을 쌍으로 맺는 일
- 파노라마, 물체 인식, 물체 추적, 스테레오 비전, 카메라 캘리브레이션 등에 필수
 - 예) 파노라마 영상 제작에서 봉합되는 지점을 결정하는데 필수



파노라마 : 여러 시점을 하나의 넓은 시야로 합친 영상

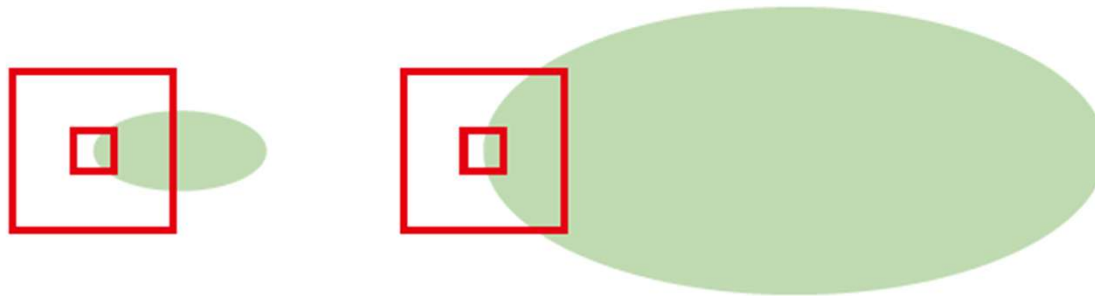
특징 검출

- 좋은 특징(feature)의 조건
 - 불변성(Invariance)
 - 크기, 회전, 각도, 조명등의 변화에도 특징의 값이 크게 변하지 않아야 함
 - 변별성(Distinctiveness)
 - 코너나 패턴이 많은 영역처럼 다른 영역과 명확히 달라야 함
 - 재현력(Repeatability)
 - 다른 조건(다른 영상)에서도 특징으로 검출될 수 있어야 함
 - 지역성(Locality)
 - 특징점이 영상의 좁고 국소적인 영역의 정보만을 바탕으로 정의되어야 함



특징 검출

- 해리스 코너 검출(Harris corner detection) 알고리즘의 분석
 - 물체의 실제 코너 뿐 아니라 블롭(blob)에서도 검출
 - 블롭(blob) : 주변과 색상이나 밝기가 뚜렷하게 대비되는, 비슷한 성질을 가진 픽셀들의 뭉치
 - 이동과 회전에 불변
 - 스케일에는 불변 아님



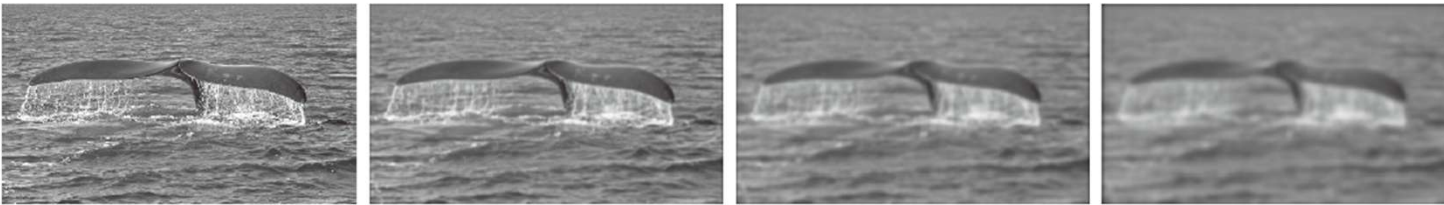
객체의 크기에 따라 적절한 마스크의 크기가 다름

특징 검출

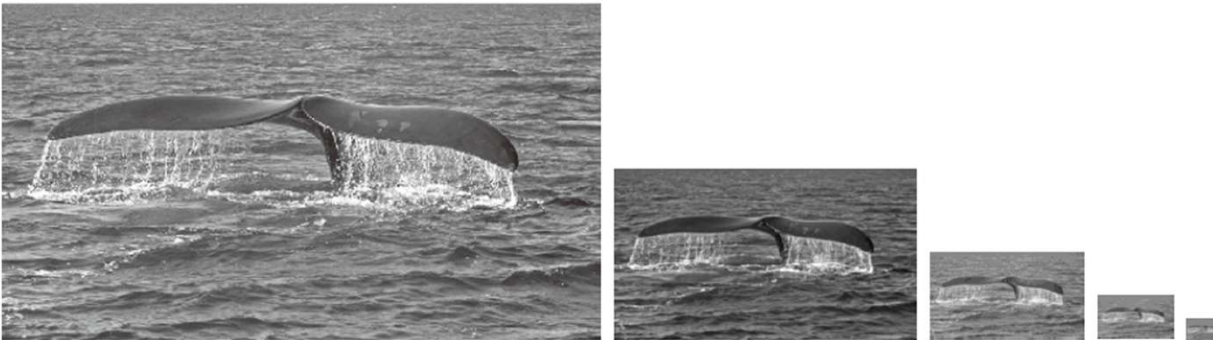
- 스케일 불변한 지역 특징
 - 사람은 스케일 불변인 특징 사용
 - 거리에 상관없이 같은 물체를 같다고 인식
 - 거리에 따라 세세한 내용에만 차이가 있음
- 스케일 공간(scale space)이론
 - 영상을 단일 해상도가 아닌, 다양한 크기(스케일)의 집합으로 다루어 사물을 인식하려는 방법
 - 영상을 단계적으로 흐리게(블러 처리) 하거나 크기를 줄여가며, 모든 스케일에서 공통적으로 발견되는 특징을 찾아냄

특징 검출

- 다중 스케일 영상 구현 방법
 - 입력은 영상 한 장. 여러 거리에서 보았을 때 나타나는 현상을 모사
 - 가우시안 스무딩과 피라미드 방법



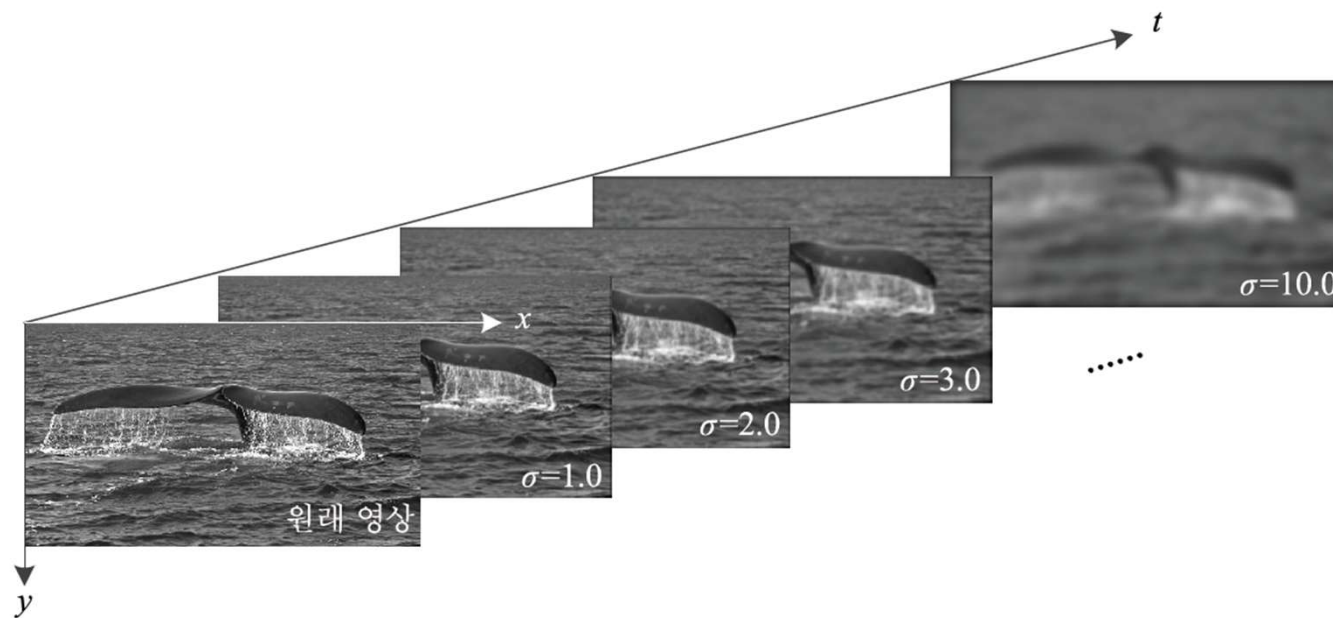
가우시안 스무딩(σ 값을 변화시키면서 스무딩)



피라미드 방법(영상의 크기를 줄임(downscaling))

특징 검출

- 다중 스케일 영상 구현 방법
 - 가우시안 스무딩은 스케일을 연속값으로 조절할 수 있는 장점이 있음



SIFT(Scale-Invariant Feature Transform)

- 이미지의 크기(Scale)가 변하거나 회전(Rotation)해도 변하지 않는 특징을 추출하고 기술(Description)하는 알고리즘
- 스케일 공간(scale space)이론에 기반하여 특징을 추출
- 일반적인 코너 검출기(Harris Corner 등)는 영상을 조금만 키우거나 돌려도 코너를 놓치는 경우가 많지만 SIFT는 다음 상황에서도 안정적인 특징을 찾음
 - 크기 변화 (Scale): 멀리서 찍든 가까이서 찍든 동일한 특징을 찾음
 - 회전 (Rotation): 카메라를 옆으로 돌려 찍어도 동일한 특징을 찾음.
 - 조명 변화 (Illumination): 사진이 조금 어둡거나 밝아져도 특징을 유지함
- 이전에는 특허권이 있어 상업적 이용이 제한적이었으나 2020년에 특허가 만료되어 자유롭게 사용 가능

SIFT(Scale-Invariant Feature Transform)

- SIFT는 크게 특징 검출(Detection) 단계와 특징 기술(Description) 단계로 나뉨
- 특징 검출 (Feature Detection)
 - 영상 내에서 크기 변화나 노이즈에 강인한 키폰트(Keypoints)를 찾는 과정
- 특징 기술 (Feature Description)
 - 찾아낸 특징을 지문처럼 고유한 숫자로 표현하여, 나중에 다른 영상에서도 매칭할 수 있게 만드는 과정

SIFT(Scale-Invariant Feature Transform)

- 특징 검출(Detection) 단계
 1. 다중 스케일 영상 구축
 - 가우시안 스무딩과 피라미드 방법을 결합해 사용
 2. 다중 스케일 영상에 미분 적용
 - 정규 라플라시안은 시간이 많이 걸려 유사한 DoG(Difference of Gaussian) 사용
 - DoG는 이전에 만든 가우시안 영상에서 이웃한 가우시안을 빼면 되기 때문에 획기적으로 빠름
 3. 극점 검출
 - DoG 영상들 사이에서 주변 26개 픽셀(같은 층 8개 + 위층 9개 + 아래층 9개)보다 값이 크거나 작은 지점을 특징 후보로 선택함으로써 스케일 불변성을 확보
 4. 키포인트 정밀화
 - 믿을만한 특징만 남기고, 그 위치를 소수점 단위(Sub-pixel)로 정확하게 조정하는 과정

SIFT(Scale-Invariant

- 특징 검출(Detection) 단계

- 다중 스케일 영상 구축

- 가우시안 스무딩과 피라미드 생성

- 다중 스케일 영상에 DoG 적용

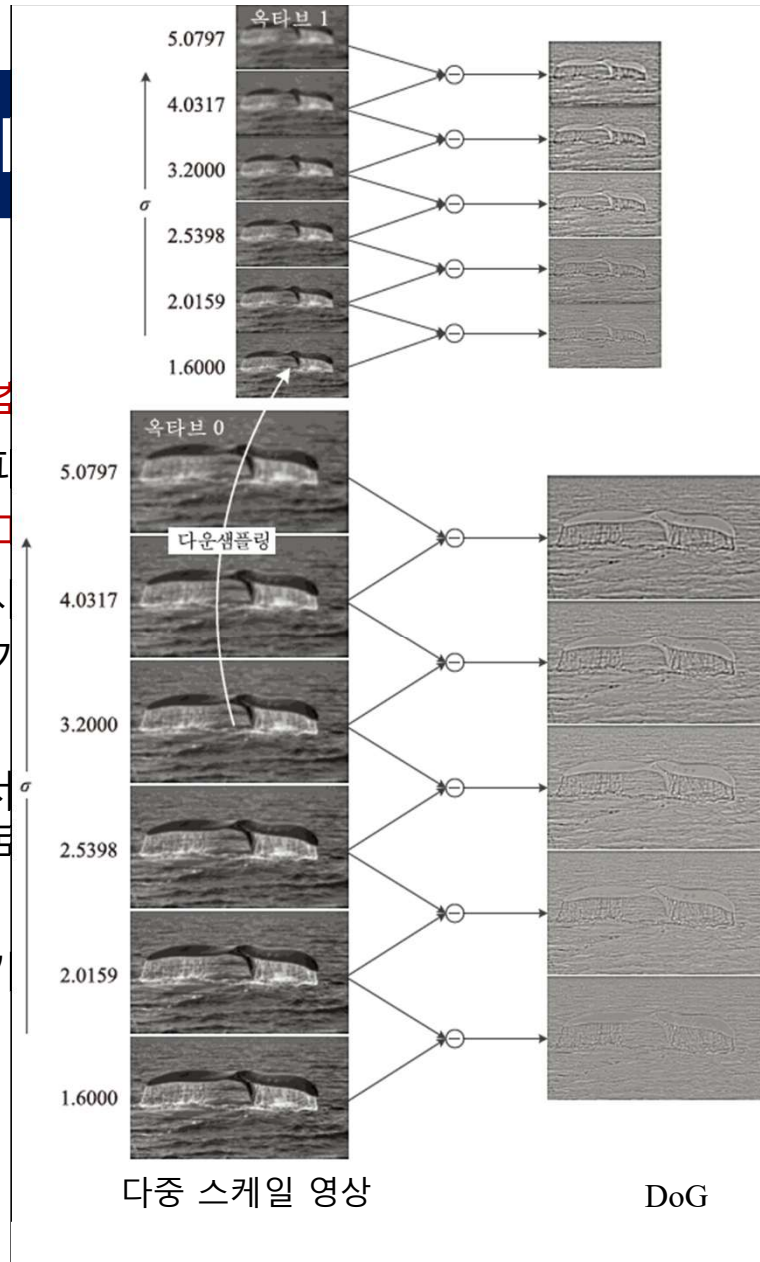
- 정규 라플라시안은 사용
- DoG는 이전에 만든 가우시안 피라미드에서 차분

- 극점 검출

- DoG 영상들 사이에서 극대/극소점을 특징 후보로

- 키포인트 정밀화

- 믿을만한 특징만 남기



(o : octave, i : interval)

(Gaussian) 사용

! 되기 때문에 획기적으로 빠름

(아래층 9개)보다 값이 크거나 작

확하게 조정하는 과정

SIFT(Scale-Invariant

- 특징 검출(Detection) 단계

- 다중 스케일 영상 구축

- 가우시안 스무딩과 피라미드 생성

- 다중 스케일 영상에 DoG 적용

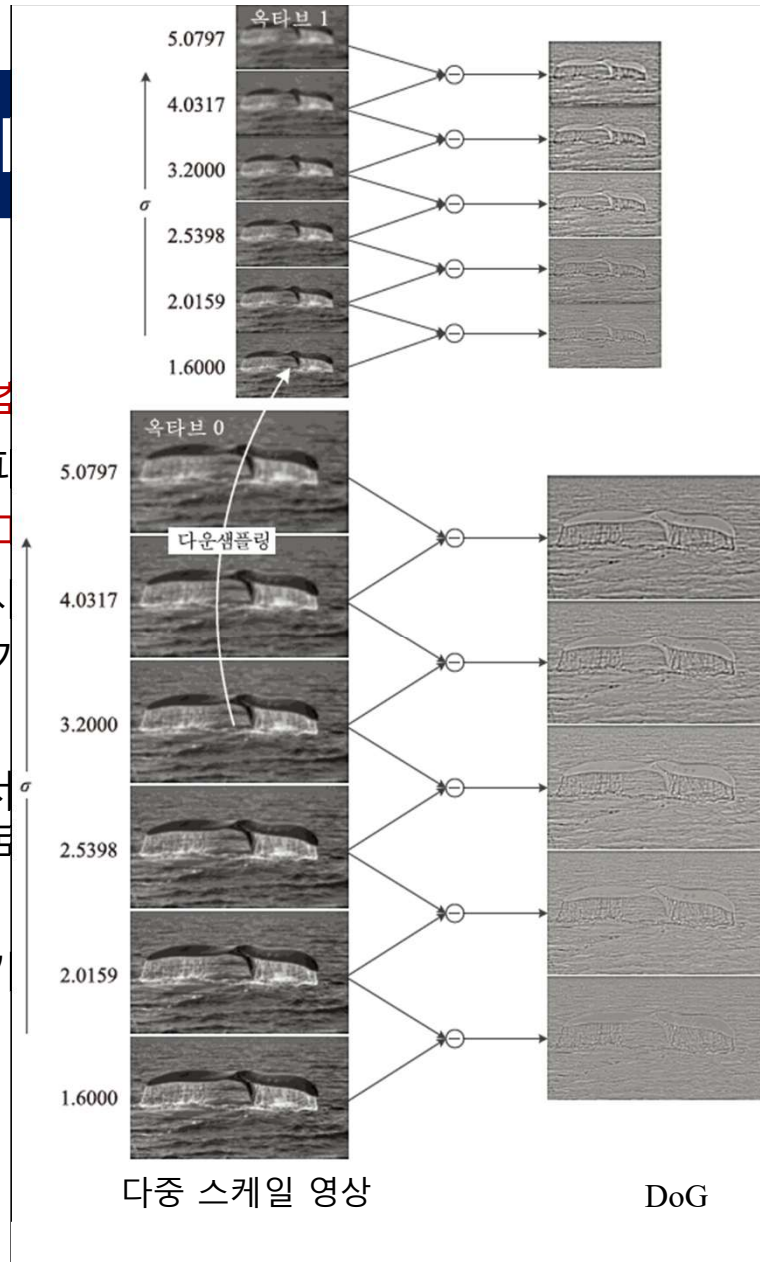
- 정규 라플라시안은 사용
- DoG는 이전에 만든 가우시안 피라미드에서 생성

- 극점 검출

- DoG 영상들 사이에서 극점을 찾아 특징 후보로 선정

- 키포인트 정밀화

- 믿을만한 특징만 남기



(o : octave, i : interval)

$o=0$ 이 원본 크기라면

$o=1$ 은 가로세로가 1/2로 줄어든 영상

$o=2$ 는 1/4로 줄어든 영상

i : 각 octave(o) 당 DoG의 층

(Gaussian) 사용

! 되기 때문에 획기적으로 빠름

(아래층 9개)보다 값이 크거나 작

확하게 조정하는 과정

SIFT(Scale-Invariant Feature Transform)

- 특징 검출(Detection) 단계

- 다중 스케일 영상 구축

- 가우시안 스무딩과 피라미드 방

- 다중 스케일 영상에 미분 적용

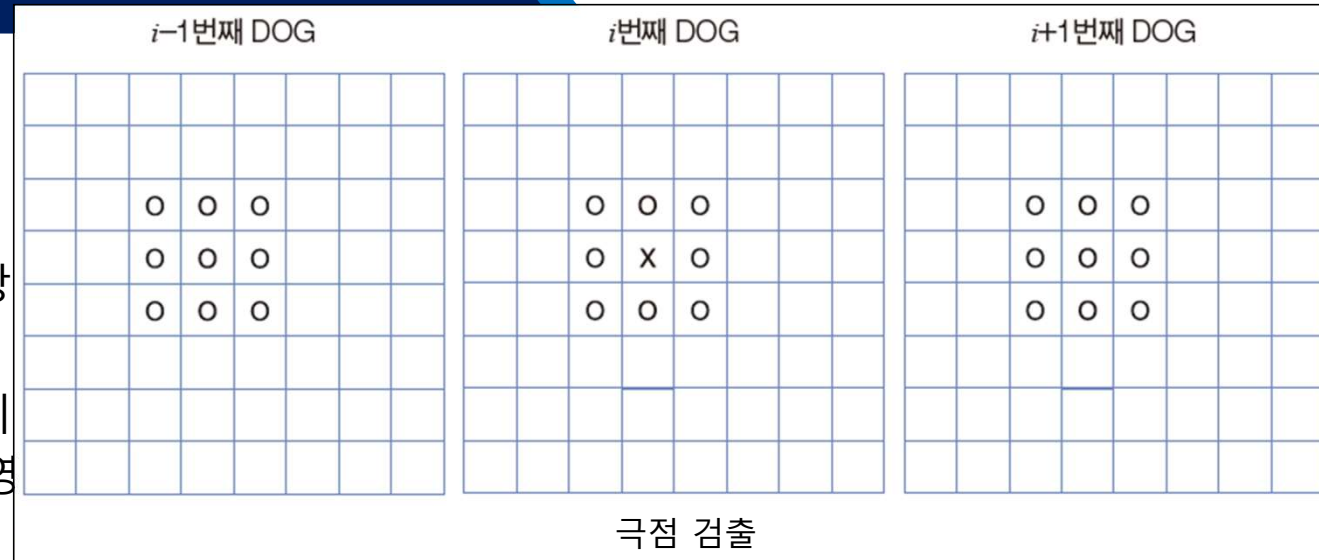
- 정규 라플라시안은 시간이 많이
- DoG는 이전에 만든 가우시안 영

- 극점 검출

- DoG 영상들 사이에서 주변 26개 픽셀(같은 층 8개 + 위층 9개 + 아래층 9개)보다 값이 크거나 작은 지점을 특징 후보로 선택함으로써 스케일 불변성을 확보

- 키포인트 정밀화

- 믿을만한 특징만 남기고, 그 위치를 소수점 단위(Sub-pixel)로 정확하게 조정하는 과정



DoG 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

# 영상 로드 및 흑백 변환

url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/butterfly.jpg'
urllib.request.urlretrieve(url, 'butterfly.jpg')

img = cv2.imread('butterfly.jpg')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)


# 스케일 공간 설정 파라미터

num_octaves = 4 # 총 4개의 옥타브 생성
sigma0 = 1.6
k = 2**(1.0/s) # 스케일 배수


# 스케일 공간 생성 (Gaussian Pyramid & DoG Pyramid)

gaussian_pyr = []
dog_pyr = []
```

```
# 원본 이미지의 초기 가우시안 필터링

base_img = cv2.GaussianBlur(gray, (0, 0), sigmaX=sigma0, sigmaY=sigma0)
gaussian_pyr.append([base_img])


for o in range(num_octaves):

    current_octave_gaussians = [base_img]
    current_octave_dogs = []

    # 하나의 옥타브 내에서 s+3개의 가우시안 이미지를 만들

    for i in range(1, s+3):

        # 이전 층의 sigma에 k를 곱해 가며 흐리게 만들.

        prev_sigma = sigma0 * (k ** (i-1))

        gaussian_img = cv2.GaussianBlur(current_octave_gaussians[-1], (0, 0),
sigmaX=prev_sigma)

        current_octave_gaussians.append(gaussian_img)


    # 인접한 가우시안 이미지의 차이 (DoG) 계산(high_sigma - low_sigma 순서)

    dog_img = current_octave_gaussians[i] - current_octave_gaussians[i-1]
    current_octave_dogs.append(dog_img)
```

DoG 실습

```
gaussian_pyr.append(current_octave_gaussians)
dog_pyr.append(current_octave_dogs)

# 다음 옥타브를 위해 크기를 절반으로 줄임
base_img = cv2.resize(current_octave_gaussians[s], (0, 0), fx=0.5, fy=0.5,
interpolation=cv2.INTER_NEAREST)

# 결과 시각화
print(f'--- SIFT DoG Pyramid (Octaves: {num_octaves}, Intervals: {s}) ---')
for o in range(num_octaves):
    num_dogs = len(dog_pyr[o])
    viz_list = []
    for d_img in dog_pyr[o]:
        # 차이값이 매우 작으므로 잘 보이게 정규화
        norm_img = cv2.normalize(d_img, None, 0, 255, cv2.NORM_MINMAX)
        viz_list.append(norm_img)

octave_viz = np.hstack(viz_list)
print(f'\n[Octave {o}] (Size: {octave_viz.shape[1]}x{octave_viz.shape[0]})')
cv2_imshow(octave_viz)
```


과제 #1

1. 현재까지 실습 코드(DoG까지)를 모두 수행하고 결과 출력 (1점)
 - 각 수행마다 코드에는 어떤 과정인지 주석 처리
2. 영상을 영상의 중심(height/2, width/2)을 축으로 임의의 각도(θ)만큼 회전 및 (a, b)만큼 이동 하는 프로그램(1.5점, 오실행 1점, 코드만 제출 0.5점)
 - Butterfly 영상을 입력으로 실행
 - Butterfly 영상을 225도 만큼 회전하고 (80, 80)만큼 이동한 결과가 출력되도록 할 것
 - 원래의 영상 크기보다 변환공간을 충분히 넓게 잡아야 회전했을 때 영상이 잘리지 않음
 - 영상의 기하 변환은 픽셀단위로 수행되어야 함(Backward mapping 방식으로 수행, OpenGL 변환함수 변환X)
 - 각 수행마다 코드에는 어떤 과정인지 주석 처리

제출 방법 : LMS로 압축 파일 제출

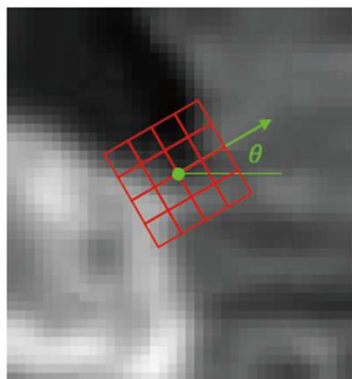
- 코랩에서 다음과 같은 이름을 가진 노트북 파일을 생성한다.
ComputerVision_1stHW_Class본인소속반_학번_영문이름.ipynb
예) ComputerVision_1stHW_Class1_24xxxxx_HanshinLim.ipynb
- 생성된 코랩 노트북 파일(.ipynb)에서 코드를 구현 및 결과를 출력한다. 결과 출력시 어떤 결과인지를 명시한다.
- Due : 4월 8일 밤 12시 (이후 제출 0점)

SIFT(Scale-Invariant Feature Transform)

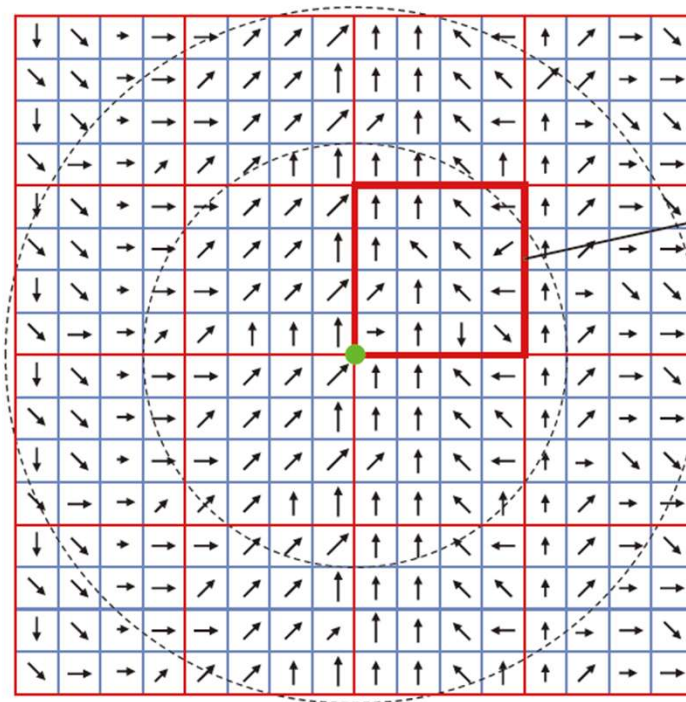
- 특징 기술(Description) 단계
 - 특징점 주위를 살펴 풍부한 정보를 가진 기술자 추출
 - 불변성 달성이 매우 중요
 - 기술자 추출 알고리즘
 1. o 와 i 로 가장 가까운 가우시안을 결정하고 거기서 기술자 추출(스케일 불변성 달성)
 2. 기준 방향을 정하고 기준 방향을 중심으로 특징을 추출(회전 불변성 달성)
 3. 기술자 x 를 단위 벡터로 바꿈(조명 불변성 달성)

SIFT(Scale-Invariant Feature Transform)

- SIFT 기술자 추출



지배적인 방향의 윈도우



16x16 부분 영역 샘플링과 기술자 추출

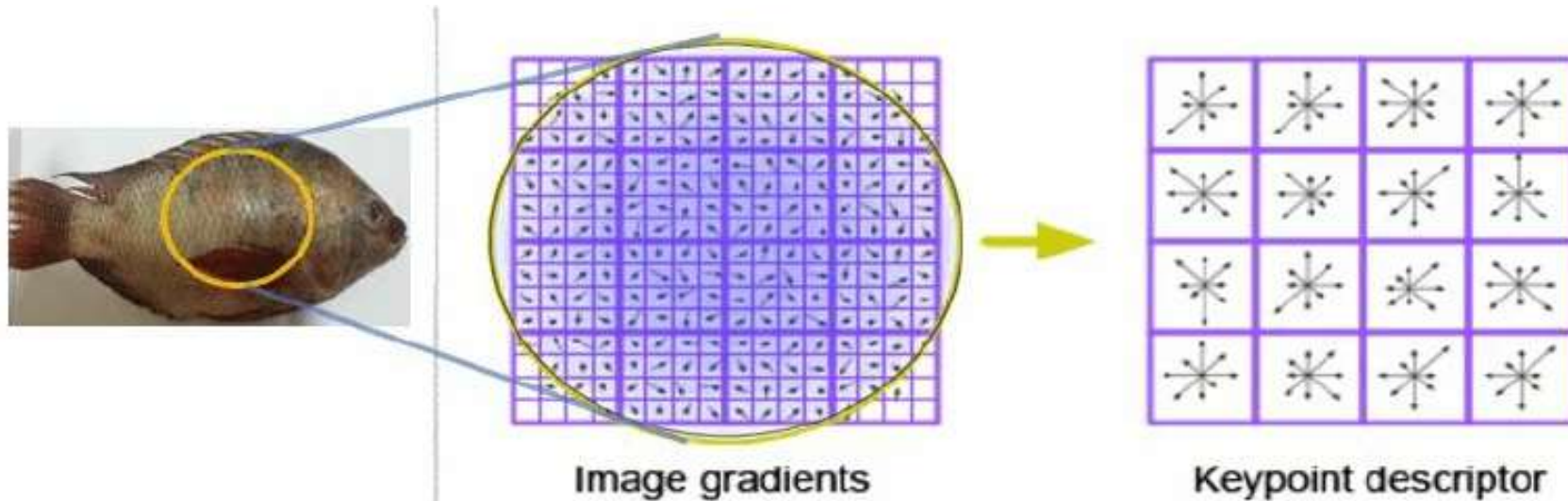
최종 $16 \times 8 = 128$ 차원의 기술자를 얻음

● 특징점의 중심 (y, x)

○ 가우시안 가중치

SIFT(Scale-Invariant Feature Transform)

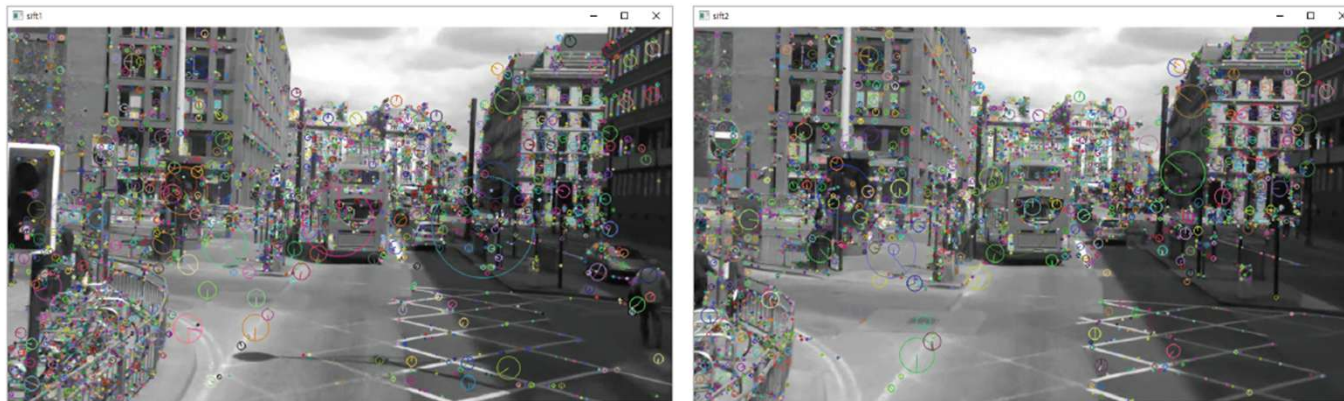
- SIFT 기술자 추출



SIFT 기술자 추출 예시

매칭(Matching)

- 매칭 문제의 이해
 - 두 영상에서 추출한 기술자 집합 $A=\{a_1, a_2, \dots, a_m\}$ 와 $B=\{b_1, b_2, \dots, b_n\}$ 에서 같은 물체의 같은 곳에서 추출된 쌍을 모두 찾는 문제
 - 매칭을 적용하는 다양한 상황
 - 물체 인식에서는 물체의 모델 영상이 A 이고 장면 영상이 B
 - 물체 추적이나 스테레오는 두 영상이 동등한 입장
- 매칭 문제의 어려움
 - 두 영상 모두 특징이 많아 후보 쌍이 아주 많은
 - 기술자에 잡음이 섞여있음



매칭(Matching)

- 두 기술자(descriptor) $\mathbf{a}=[a_1, \dots, a_d]$, $\mathbf{b}=[b_1, \dots, b_d]$ 간 거리 계산 (SIFT의 경우 $d=128$)
 - 유클리디안(Euclidean, L2) 거리

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{k=1, d} (a_k - b_k)^2} = \|\mathbf{a} - \mathbf{b}\| \quad (5.12)$$

- 매칭 전략

- 최근접 이웃: $\mathbf{a}_i$ 는 B 에서 가장 가까운 $\mathbf{b}_j$ 를 찾고 $\mathbf{a}_i$ 와 $\mathbf{b}_j$ 가 아래 식을 만족시키면 매칭쌍으로 간주

$$d(\mathbf{a}_i, \mathbf{b}_j) < T \quad T: \text{임계값}$$

- 최근접 이웃 거리 비율: $\mathbf{a}_i$ 는 B 에서 가장 가까운 $\mathbf{b}_j$ 와 두 번째로 가까운 $\mathbf{b}_k$ 를 찾음. $\mathbf{b}_j$ 와 $\mathbf{b}_k$ 가 아래 식을 만족시키면 $\mathbf{a}_i$ 와 $\mathbf{b}_j$ 는 매칭쌍으로 간주

$$\frac{d(\mathbf{a}_i, \mathbf{b}_j)}{d(\mathbf{a}_i, \mathbf{b}_k)} < T \quad T: \text{임계값}$$

매칭(Matching)

- 두 기술자(descriptor) $\mathbf{a}=[a_1, \dots, a_d]$, $\mathbf{b}=[b_1, \dots, b_d]$ 간 거리 계산 (SIFT의 경우 $d=128$)
 - 유클리디안(Euclidean, L2) 거리

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{k=1, d} (a_k - b_k)^2} = \|\mathbf{a} - \mathbf{b}\|$$

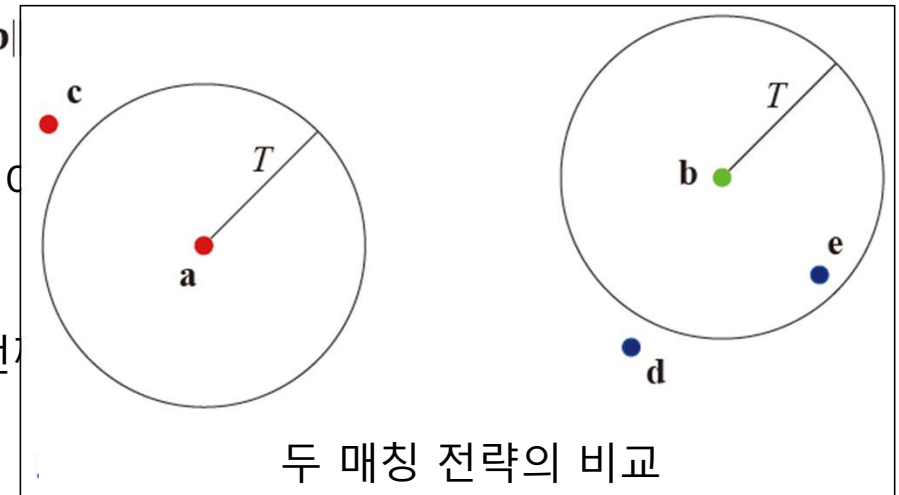
- 매칭 전략

- 최근접 이웃: $\mathbf{a}_i$ 는 B 에서 가장 가까운 $\mathbf{b}_j$ 를 찾고 $\mathbf{a}_i$ 와 $\mathbf{b}_j$ 가

$$d(\mathbf{a}_i, \mathbf{b}_j) < T \quad T: \text{임계값}$$

- 최근접 이웃 거리 비율: $\mathbf{a}_i$ 는 B 에서 가장 가까운 $\mathbf{b}_j$ 와 두 번만족시키면 $\mathbf{a}_i$ 와 $\mathbf{b}_j$ 는 매칭쌍으로 간주

$$\frac{d(\mathbf{a}_i, \mathbf{b}_j)}{d(\mathbf{a}_i, \mathbf{b}_k)} < T \quad T: \text{임계값}$$



SIFT도 최근접 이웃 거리 비율 전략을 사용
(Lowe's Ratio Test)

매칭 성능 측정

- 정확도(Accuracy)와 F1-Score

- 같은 색이 실제 매칭쌍(정답, GT)이고 연결선이 매칭 알고리즘이 맺어준 쌍(예측)이라고 했을 때



- 혼동 행렬(confusion matrix)

		정답(GT)	
		긍정	부정
예측	긍정	참 긍정(TP)	거짓 긍정(FP)
	부정	거짓 부정(FN)	참 부정(TN)

정확도 (Accuracy)

- 전체 데이터 중 정답으로 분류되는 비율

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

매칭 성능 측정

- 정확도(Accuracy)와 F1-Score

- 같은 색이 실제 매칭쌍(정답, GT)이고 연결선이 매칭 알고리즘이 맺어준 쌍(예측)이라고 했을 때



- 혼동 행렬(confusion matrix)

		정답(GT)	
		긍정	부정
예측	긍정	참 긍정(TP)	거짓 긍정(FP)
	부정	거짓 부정(FN)	참 부정(TN)

정밀도 (Precision)

- 매칭으로 예측했을 때 실제로 매칭인 비율

$$Precision = \frac{TP}{TP+FP}$$

재현율 (Recall, Sensitivity)

- 실제 매칭들 중 매칭으로 예측되는 비율

$$Recall = \frac{TP}{TP+FN}$$

F1-Score (F-Score)

- 정밀도와 재현율의 조화평균

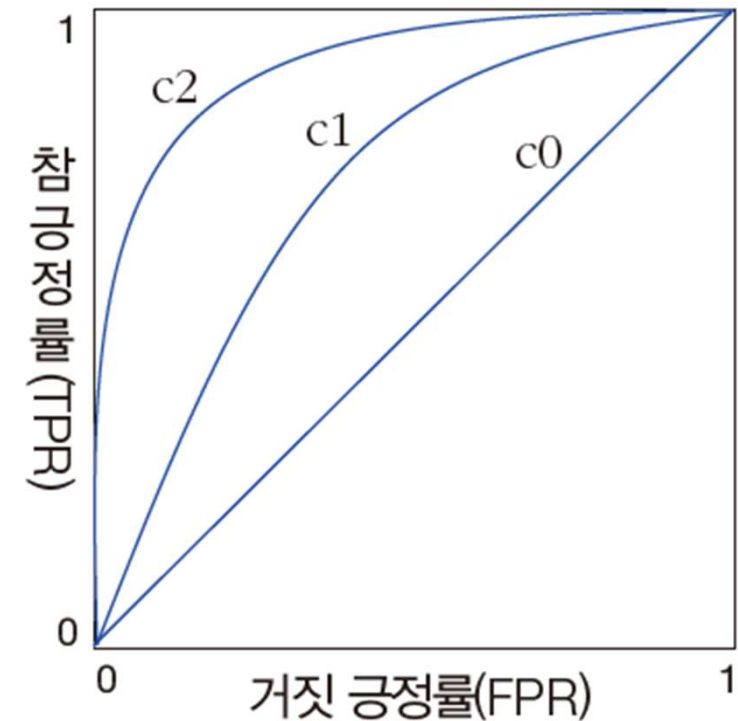
$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

매칭 성능 측정

- ROC(Receiver Operating Characteristic) 곡선
 - 아래 식에서 T 를 작게 하면 거짓 긍정률(FPR, False Positive Rate)은 작아짐
 - T 를 크게 하면 거짓 긍정률이 커지는데 참 긍정률 (TPR, True Positive Rate) 도 따라 커지는 경향이 있음

$$\frac{d(\mathbf{a}_i, \mathbf{b}_j)}{d(\mathbf{a}_i, \mathbf{b}_k)} < T \quad T: \text{임계값}$$

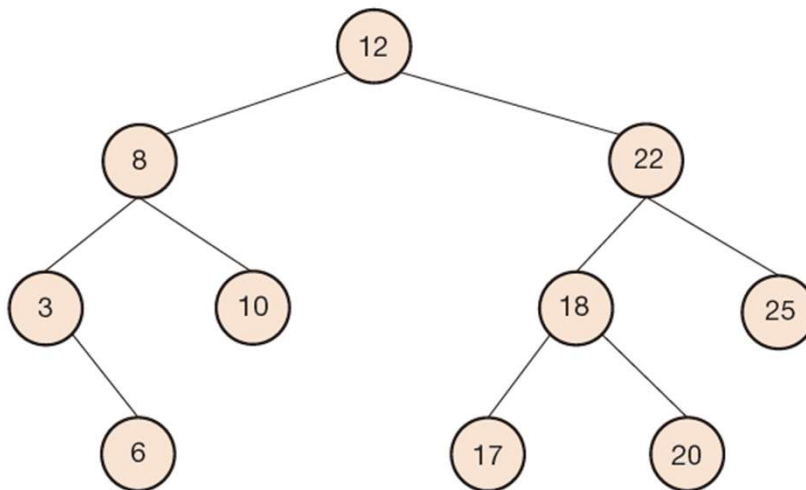
$$\left. \begin{aligned} \text{참 긍정률} &= \frac{TP}{TP + FN} \\ \text{거짓 긍정률} &= \frac{FP}{TN + FP} \end{aligned} \right\} \quad (5.17)$$



c2가 가장 좋은 알고리즘

빠른 매칭

- 빠른 탐색을 위한 자료 구조를 도입해 사용
 - 이진 탐색 트리(Binary search tree)와 해싱(Hashing) 등



이진 탐색 트리(Binary search tree)

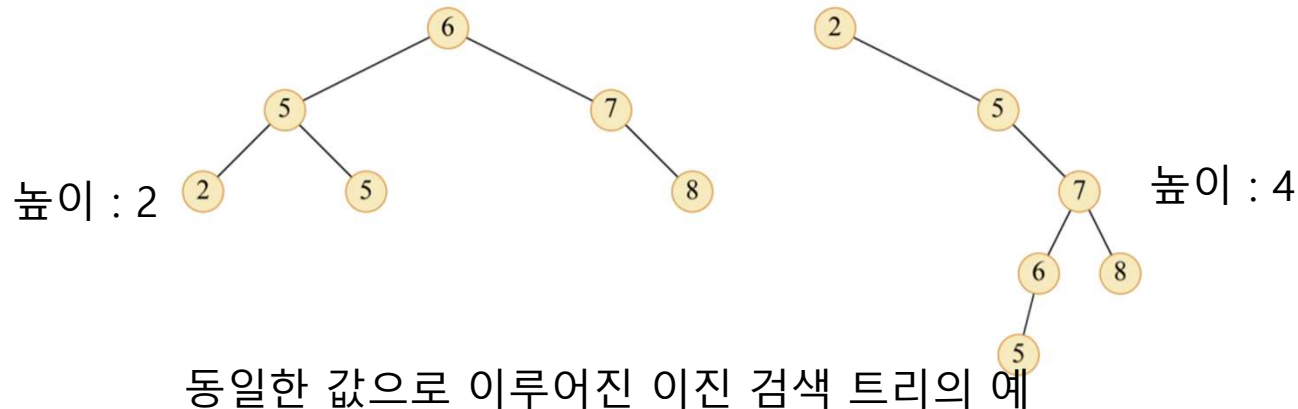
해시 함수
 $h(x) = x \% 13$

0	
1	14
2	
3	
4	134
5	
6	
7	7
8	
9	
10	65023
11	
12	

해싱(Hashing)

빠른 매칭

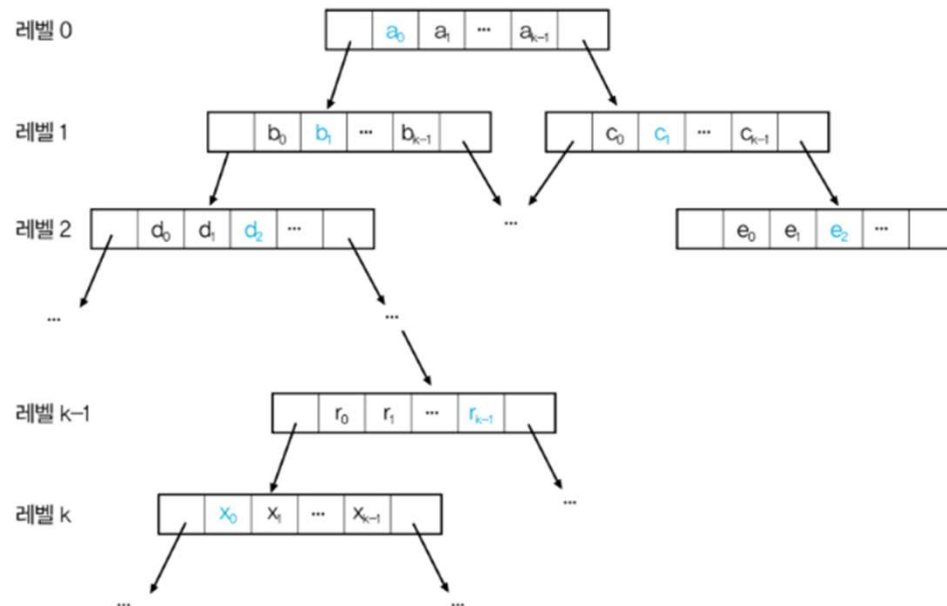
- 검색 트리(Search tree)
 - 데이터를 트리 구조로 저장하여 빠른 탐색 및 저장 등이 가능하도록 만든 자료 구조
- 이진 탐색 트리(Binary search tree)
 - 최상위 레벨에 루트 노드가 있고 한 노드에서 최대 2개의 자식 노드를 가짐
 - 이진 검색 트리에 대한 기본 연산은 트리의 높이에 비례하는 시간에 수행
 - 임의의 노드의 값은 자신의 왼쪽에 있는 모든 노드의 값보다 크고 오른쪽에 있는 모든 노드의 값보다 작음
 - 값들의 삽입 순서에 따라 모양이 결정



빠른 매칭

- kd 트리

- 특징의 기술자(descriptor)는 벡터이기 때문에 이진 탐색 트리 그대로 적용이 불가능
- k 차원 공간의 점들을 효율적으로 저장하고 탐색하기 위한 이진 공간 분할(binary space partitioning) 트리
- 이진 검색 트리를 확장한 것으로 k 개($k \geq 2$)의 필드(차원)로 이루어지는 벡터를 사용
- 검색 트리의 각 레벨이 하나씩의 차원만 다름

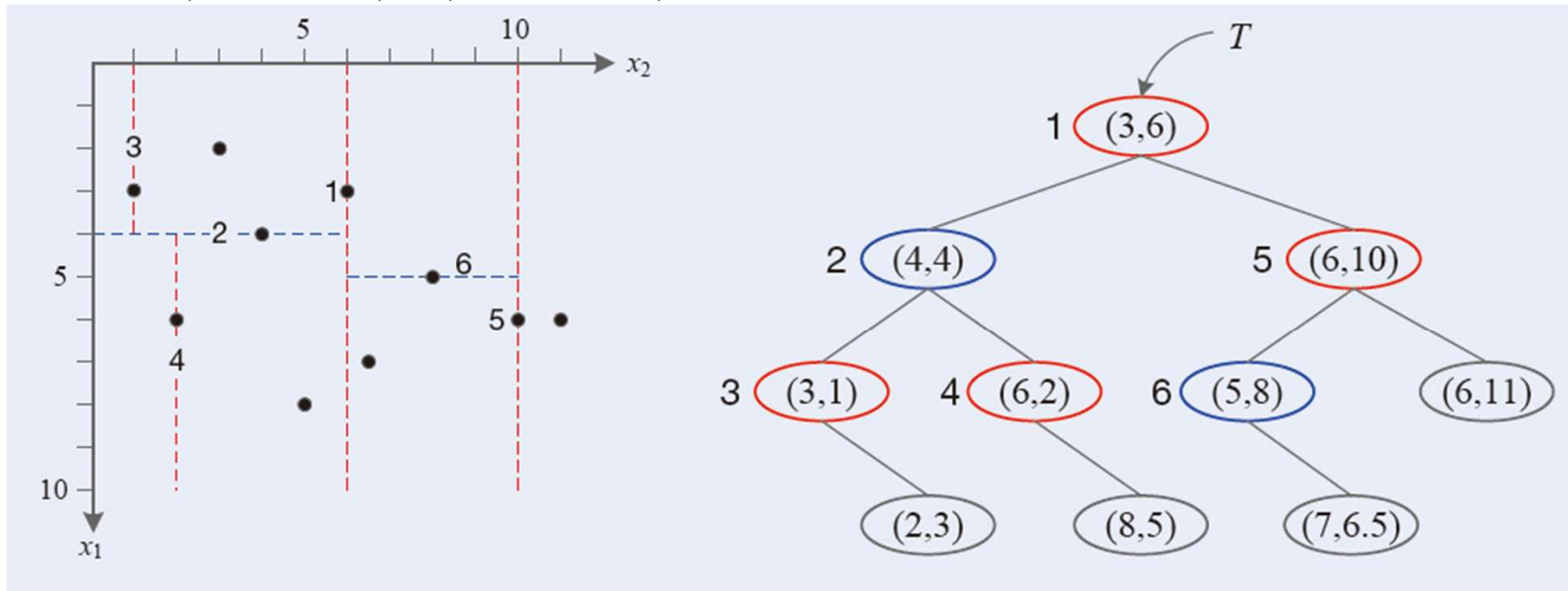


빠른 매칭

- kd 트리 만들기

$X = \{x_1 = (3,1), x_2 = (2,3), x_3 = (6,2), x_4 = (4,4), x_5 = (3,6), x_6 = (8,5), x_7 = (7,6.5), x_8 = (5,8), x_9 = (6,10), x_{10} = (6,11)\}$

- 두 축 값은 (3,2,6,4,...,6)과 (1,3,2,4,...,11). 두 번째 축의 분산이 더 커서 두 번째 축 선택

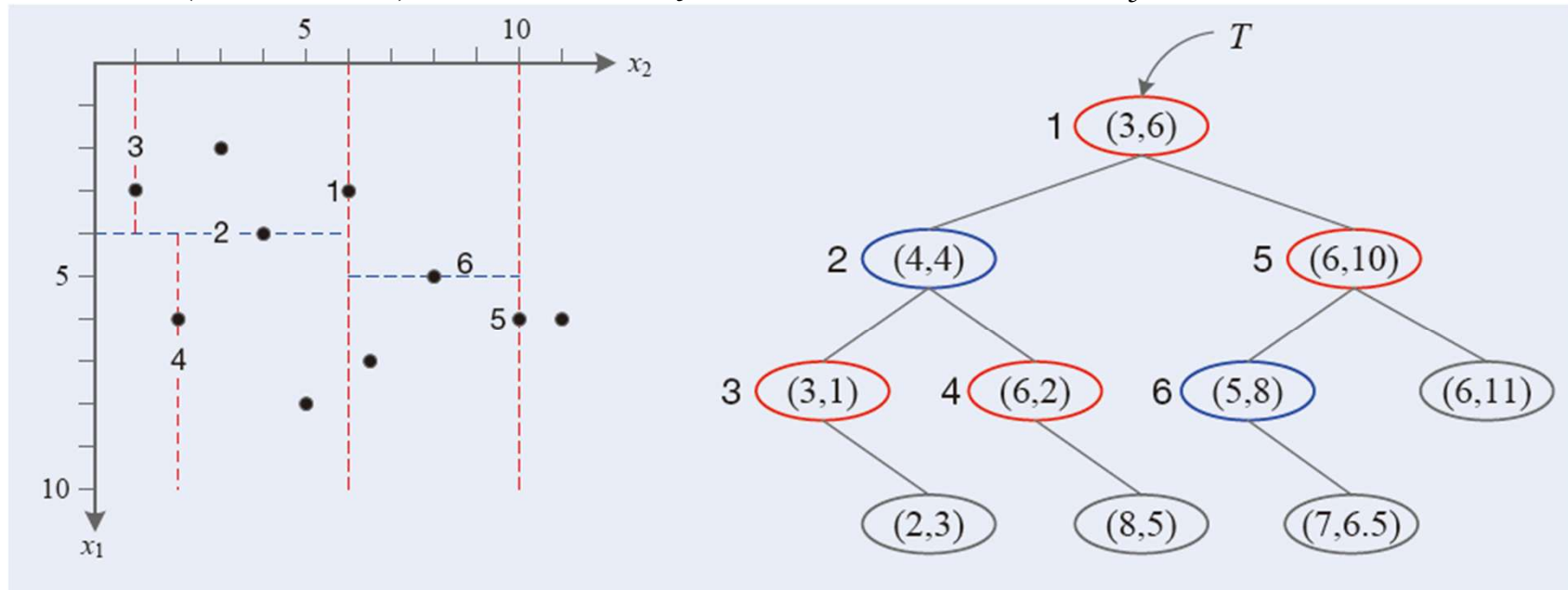


빠른 매칭

- kd 트리 만들기

$X = \{x_1 = (3,1), x_2 = (2,3), x_3 = (6,2), x_4 = (4,4), x_5 = (3,6), x_6 = (8,5), x_7 = (7,6.5), x_8 = (5,8), x_9 = (6,10), x_{10} = (6,11)\}$

- 두 번째 축 (1,3,2,4,...,11) 정렬. 중앙값은 $j=5$, 즉 다섯 번째 기술자 x_5 가 분할 기준

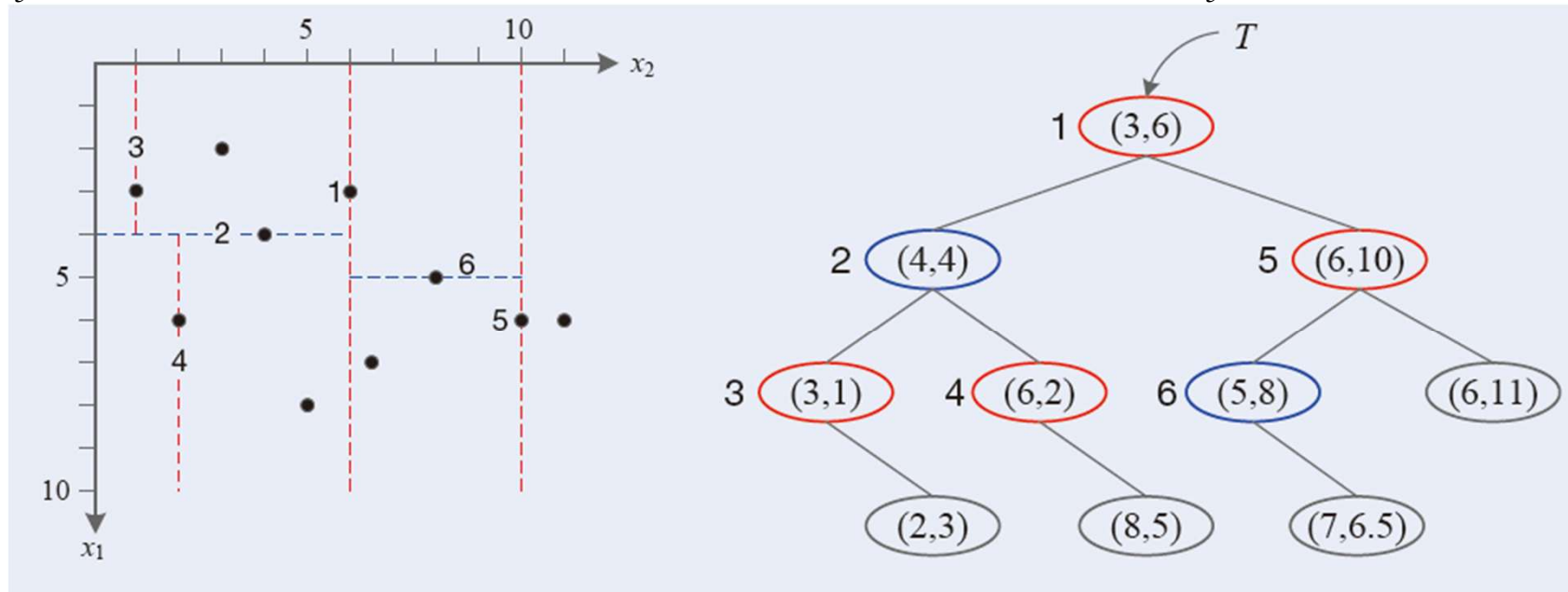


빠른 매칭

- kd 트리 만들기

$X = \{x_1 = (3,1), x_2 = (2,3), x_3 = (6,2), x_4 = (4,4), x_5 = (3,6), x_6 = (8,5), x_7 = (7,6.5), x_8 = (5,8), x_9 = (6,10), x_{10} = (6,11)\}$

- x_5 와 두 번째 축을 기준으로 나머지 9개 기술자를 분할 및 루트 노드에 x_5 를 배치하고 분할 정보(축) 저장

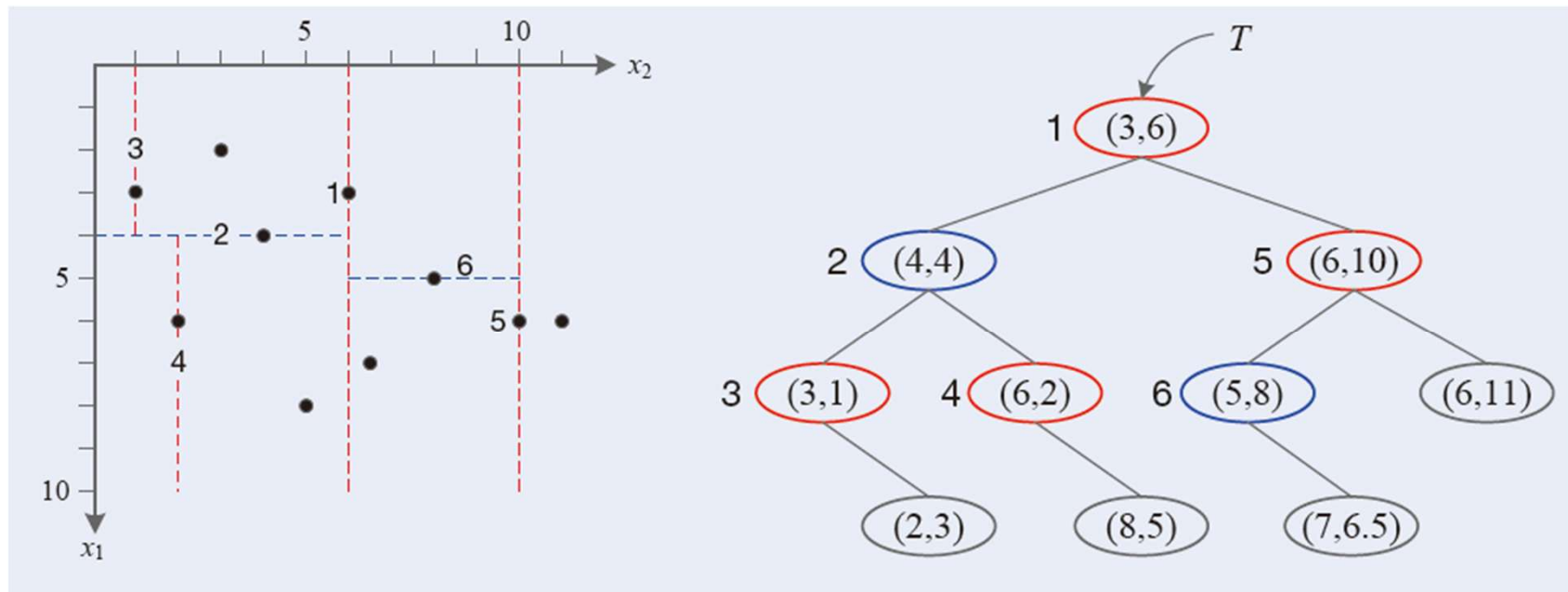


빠른 매칭

- kd 트리 만들기

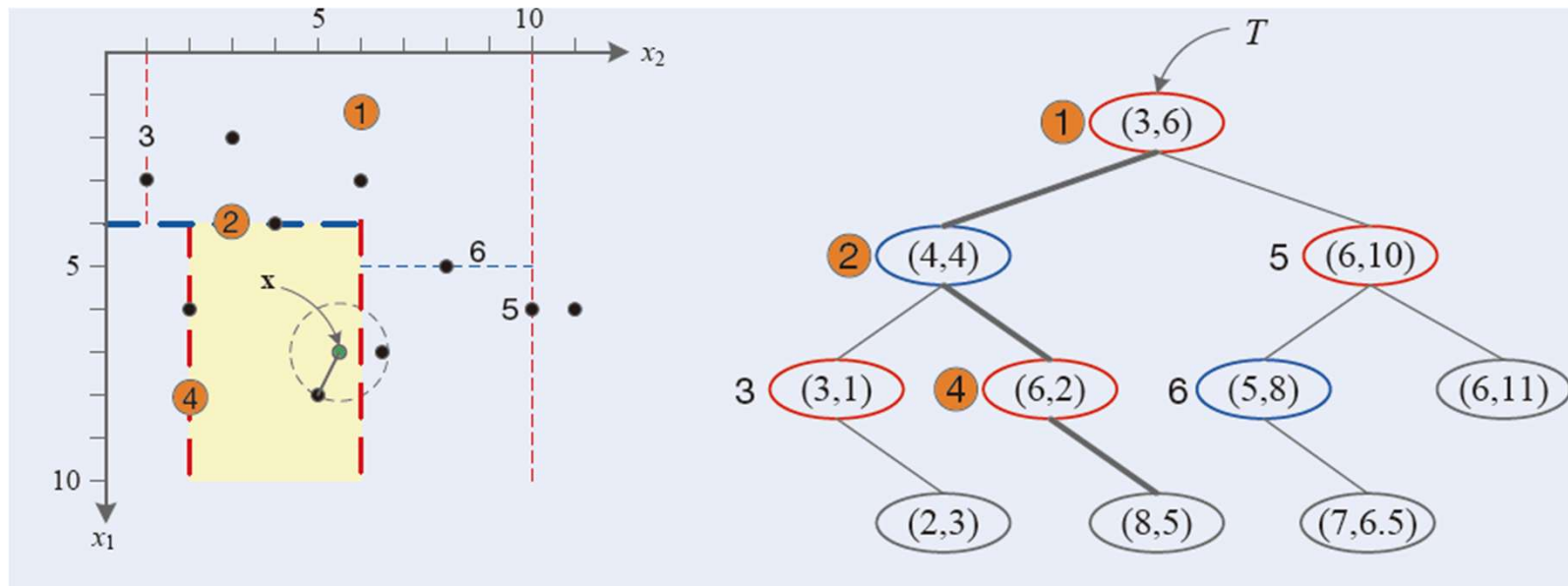
$X = \{x_1 = (3,1), x_2 = (2,3), x_3 = (6,2), x_4 = (4,4), x_5 = (3,6), x_6 = (8,5), x_7 = (7,6.5), x_8 = (5,8), x_9 = (6,10), x_{10} = (6,11)\}$

- 왼쪽 및 오른쪽 기술자들에 대해 같은 과정을 재귀적으로 반복



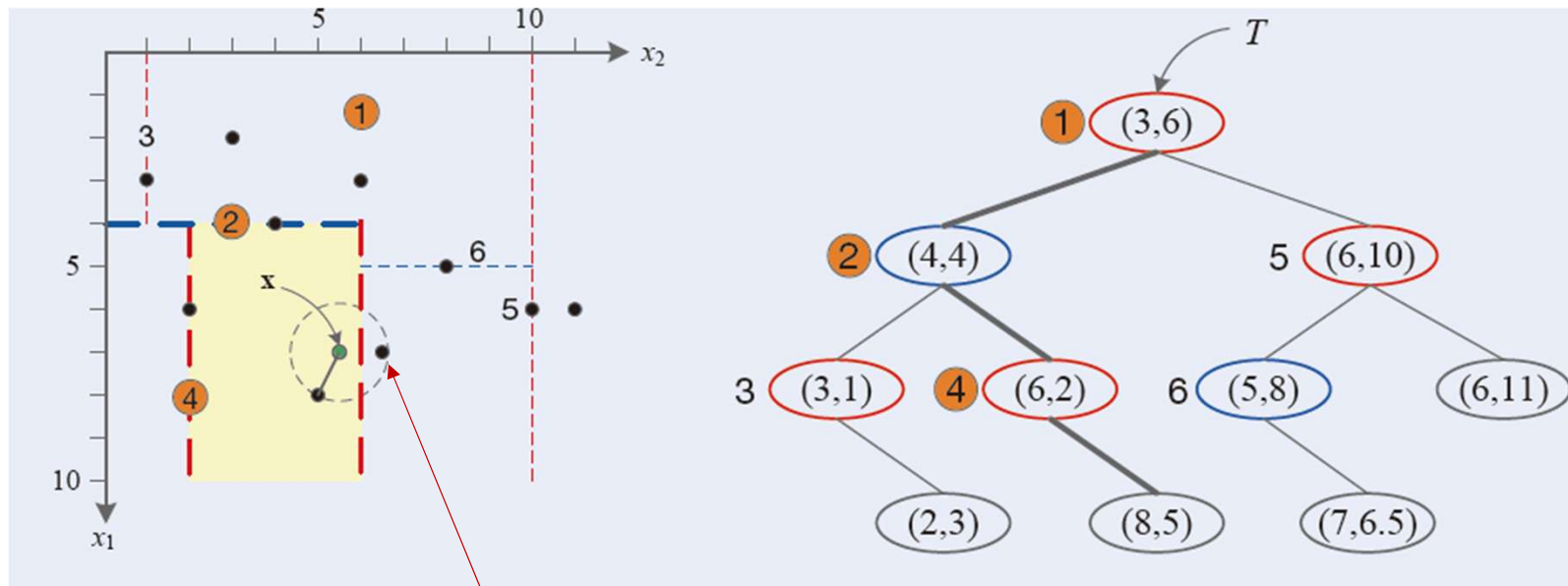
빠른 매칭

- kd 트리를 이용한 최근접 탐색
 - 새로운 특징 벡터 $\mathbf{x}=(7, 5.5)$ 가 입력되었다고 가정하고 최근접 이웃을 찾는 과정



빠른 매칭

- kd 트리를 이용한 최근접 탐색
 - 새로운 특징 벡터 $\mathbf{x}=(7, 5.5)$ 가 입력되었다고 가정하고 최근접 이웃을 찾는 과정



kd 트리 탐색

더 가까운 점 -> 타고 내려온 경로를 역추적하여 더 가까운 점을 찾는 추가 과정 필요

빠른 매칭

- FLANN(Fast Library for Approximate Nearest Neighbors)
 - 근사 최근접 이웃(Approximate Nearest Neighbor, ANN) 탐색 라이브러리
 - 사용자가 최적의 알고리즘을 고를 필요 없이 데이터의 특성에 맞춰 자동으로 가장 빠른 탐색 알고리즘(kd Tree, KMeans Tree 등)을 선택해 주는 기능
 - OpenCV에 내장되어 있어 접근성이 높음
 - 설정이 간편하고 SIFT 같은 전통적인 영상 간 특징점 매칭에 최적화

SIFT 실습

```
import cv2
import numpy as np
import urllib.request
from google.colab.patches import cv2_imshow

# 이미지 다운로드
url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/butterfly.jpg'
urllib.request.urlretrieve(url, 'butterfly.jpg')
img = cv2.imread('butterfly.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 비교를 위해 이미지를 강제로 회전 및 축소시킴
rows, cols = gray.shape
M = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 0.7) # 45도 회전, 0.7배 축소
img_varied = cv2.warpAffine(gray, M, (cols, rows))

# SIFT 객체 생성 및 특징점(Keypoints) 검출
sift = cv2.SIFT_create()
```

```
# kp: 특징점 위치/크기 정보, des: 특징점을 설명하는 128차원 벡터(Descriptor)
kp1, des1 = sift.detectAndCompute(gray, None)
kp2, des2 = sift.detectAndCompute(img_varied, None)

# 특징점 매칭 (FLANN Matcher 사용)
# trees=5: 5개의 무작위 KD-트리를 병렬로 만들어 탐색 속도를 높임
# checks=50: 트리를 얼마나 깊게 뒤져볼지 결정
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)

flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des1, des2, k=2) # 가장 가까운 2개 특징 찾기

# 좋은 매칭점만 선별 (Lowe's ratio test)
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)
```

SIFT 실습

```
# 결과 시각화
# 두 이미지 사이의 매칭 라인을 그림
img_match = cv2.drawMatches(gray, kp1, img_varied, kp2, good_matches, None,
                             flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

print(f'검출된 특징점 개수: {len(kp1)}')
print(f'매칭 성공 개수: {len(good_matches)}')
cv2.imshow(img_match)
```